Experiment 3:

Develop a Map Reduce program that mines weather data and displays appropriate messages indicating the weather conditions of the day.

Program:

Main Class(WeatherAnalysis.class):

```
import org.apache.hadoop.conf.Configured;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.FileInputFormat;
import org.apache.hadoop.mapred.FileOutputFormat;
import org.apache.hadoop.mapred.JobClient;
import org.apache.hadoop.mapred.JobConf;
import org.apache.hadoop.util.Tool;
import org.apache.hadoop.util.ToolRunner;

public class WeatherAnalysis extends Configured implements Tool {

	@Override
	public int run(String[] args) throws Exception {

	if(args.length<2)
	{
		System.out.println("Plz Give Input Output Directory Correctly");
		return -1;
	}

	JobConf conf = new JobConf(WeatherAnalysis.class);
	FileInputFormat.setInputPaths(conf,new Path(args[0]));
	FileOutputFormat.setOutputPath(conf, new Path(args[1]));
	conf.setMapperClass(WeatherMapper.class);
	conf.setReducerClass(WeatherReducer.class);
	conf.setMapOutputKeyClass(Text.class);
	conf.setMapOutputValueClass(IntWritable.class);
	conf.setOutputKeyClass(Text.class);
	conf.setOutputValueClass(IntWritable.class);
	JobClient.runJob(conf);
	return 0;
	}

	public static void main(String args[]) throws Exception
	{
		int exitcode = ToolRunner.run(new WeatherAnalysis(), args);
		System.exit(exitcode);
	}

}
```

Mapper Class(WeatherMapper.class):

```
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.MapReduceBase;
import org.apache.hadoop.mapred.Mapper;
import org.apache.hadoop.mapred.OutputCollector;
import org.apache.hadoop.mapred.Reporter;


public class WeatherMapper extends MapReduceBase implements
Mapper<LongWritable,Text,Text,IntWritable>
{private Text weatherCondition = new Text();
private IntWritable temperature = new IntWritable();
        public void map(LongWritable key, Text value,
                        OutputCollector<Text, IntWritable> output, Reporter r)
                        throws IOException
        {
                String[] fields = value.toString().split(",");
      if (fields.length == 3) {
        try {
          // String date = fields[0];
           int temp = Integer.parseInt(fields[2].trim());
           temperature.set(temp);

           if (temp <= 10) {
              weatherCondition.set("Cold");
           } else if (temp > 10 && temp <= 25) {
              weatherCondition.set("Warm");
           } else {
              weatherCondition.set("Hot");
           }
                                output.collect(weatherCondition, new IntWritable(1));
        }catch (NumberFormatException e) {
           // Ignore invalid data
        }
                }
        }

        }
```

Reducer Class(WeatherReducer.class):

```
import java.io.IOException;
import java.util.Iterator;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.MapReduceBase;
import org.apache.hadoop.mapred.OutputCollector;
import org.apache.hadoop.mapred.Reducer;
```

```
import org.apache.hadoop.mapred.Reporter;


public class WeatherReducer extends MapReduceBase implements
Reducer<Text,IntWritable,Text,Text>
{

	@Override
	public void reduce(Text key, Iterator<IntWritable> values,
				OutputCollector<Text, Text> output, Reporter arg3) throws
IOException {
					int count=0;
			while(values.hasNext())
			{
				IntWritable i= values.next();
				count+= i.get();
			}
			output.collect(key, new Text("Occurrences: " + count));
	}


}
```

Execution Procedure:

1.create java project and save with appropriate project name

2.Right click on project and create 3 java class (Mainfile, Mapper, Reducer)

3.Add Jar files-right click on project ,
a.choose build path option
b.choose configure build path
c.click on add external jars
d.path - Desktop/hadoop-java-jars - select all by pressing ctrl+A , click ok

4.Add implementation code in all three java files

---> Create jar file (right click proj name, export, jar creation)

5.commands for execution - open new terminal

6.hadoop dfs -mkdir /input - create directory to store all dataset files in HDFS-
hadoop dfs -ls /input -displays files inside input directory

7.hadoop dfs -put local file system path /input/ - copies file from local file system to HDFS

8.hadoop jar path-to-jarfile mainclass /path-to-inputfile /output

copy paste path wherever required

9.to check output there are 2 ways :

a.check output from browser --- open firefox browser type in url
localhost:50070- browse filesystem choose output directory and open part-00000 file

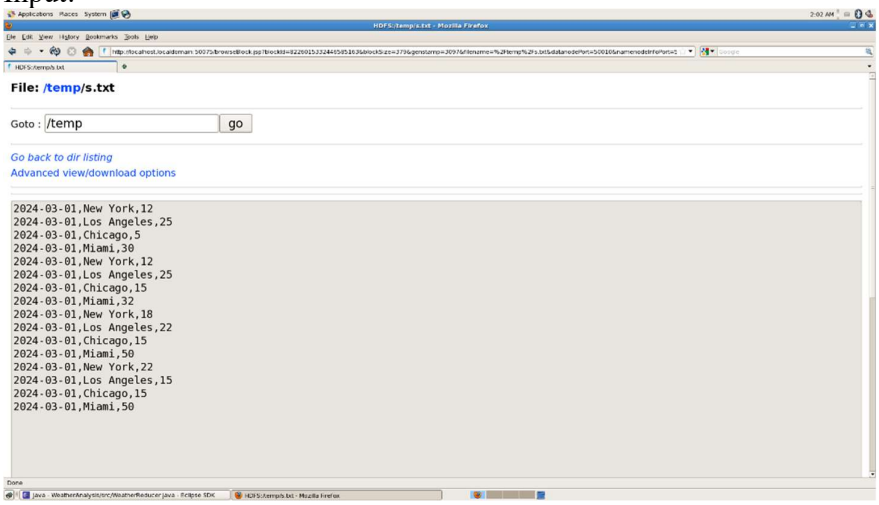
b.use cat command

hadoop dfs -ls /output --displays contents of output directory
hadoop dfs -cat /output/part-00000 - displays contents of this file

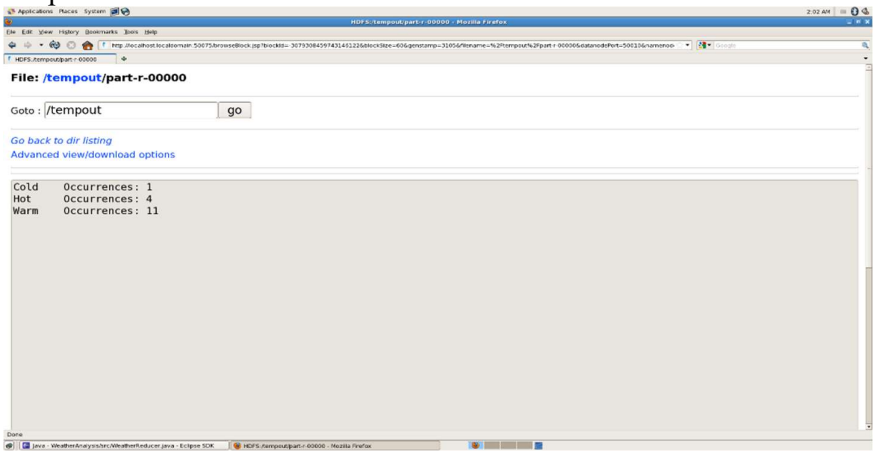
Hint: In main Class change

class name in JobConf() function
class name in setMapperClass() function
class name in setReducerClass() function
main method change in run() with constructor name

Input:



Output:



Experiment 4:

Develop a Map Reduce program to find the tags associated with each movie by analysing movie lens data.

Program:

Main Class(MovieTagsAnalysis.class):

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class MovieTagsAnalysis {
        public static void main(String[] args) throws Exception {
            Configuration conf = new Configuration();
            Job job = new Job(conf, "MovieTagsAnalysis");
            job.setJarByClass(MovieTagsAnalysis.class);
            job.setMapperClass(MovieTagMapper.class);
            job.setReducerClass(MovieTagReducer.class);

            job.setOutputKeyClass(Text.class);
            job.setOutputValueClass(Text.class);

            FileInputFormat.addInputPath(job, new Path(args[0]));
            FileOutputFormat.setOutputPath(job, new Path(args[1]));

            System.exit(job.waitForCompletion(true) ? 0 : 1);
          }
        }
```

Mapper Class(MovieTagMapper.class):

```
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;
import java.io.IOException;

public class MovieTagMapper extends Mapper<Object, Text, Text, Text> {
      private Text movieId = new Text();
      private Text tag = new Text();

      public void map(Object key, Text value, Context context) throws IOException,
InterruptedException {
        String[] fields = value.toString().split(",");

        if (fields.length >= 3 && !fields[0].equals("userId")) { // Skip header
          movieId.set(fields[1].trim()); // movieId
          tag.set(fields[2].trim()); // tag
          context.write(movieId, tag);
```

```
        }
    }
}
```

Reducer Class(MovieTagReducer.class):

```
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;
import java.io.IOException;
import java.util.HashSet;

public  class MovieTagReducer extends Reducer<Text, Text, Text, Text> {
    public  void  reduce(Text  key,  Iterable<Text>  values,  Context  context)  throws
IOException, InterruptedException {
        HashSet<String> uniqueTags = new HashSet<String>();

        for (Text val : values) {
           uniqueTags.add(val.toString());
           }

        StringBuilder tagsList = new StringBuilder();
        for (String tag : uniqueTags) {
           if (tagsList.length() > 0) {
             tagsList.append(", ");
           }
           tagsList.append(tag);
        }
        context.write(key, new Text(tagsList.toString()));
    }
}
```

Execution Procedure:

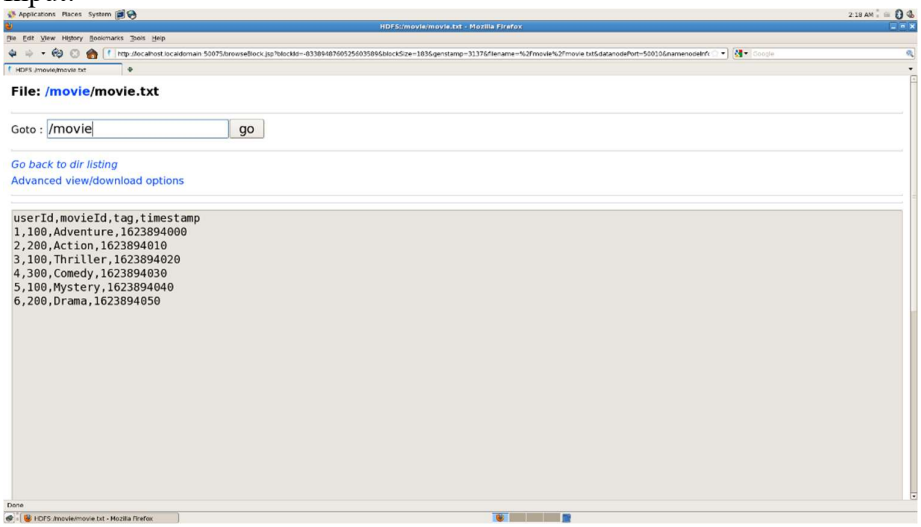
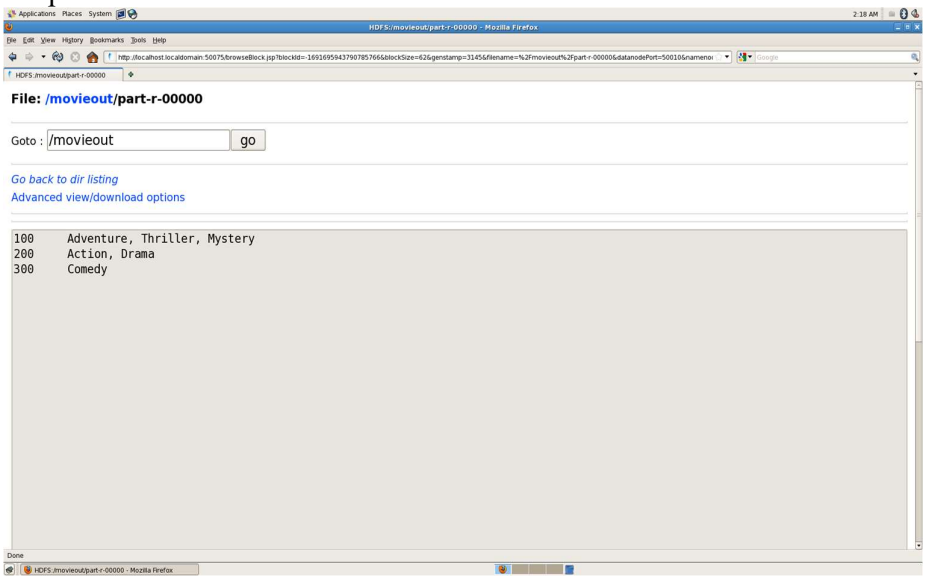
1.create java project and save with appropriate project name

2.Right click on project and create 3 java class (Mainfile, Mapper, Reducer)

3.Add Jar files-right click on project ,
a.choose build path option
b.choose configure build path
c.click on add external jars
d.path - Desktop/hadoop-java-jars - select all by pressing ctrl+A , click ok
4.Add implementation code in all three java files
 ---> Create jar file (right click proj name, export, jar creation)
5.commands for execution - open new terminal
6.hadoop dfs -mkdir /input - create directory to store all dataset files in HDFS-
  hadoop dfs -ls /input -displays files inside input directory
7.hadoop dfs -put local file system path /input/ - copies file from local file system to HDFS
8.hadoop jar path-to-jarfile mainclass /path-to-inputfile /output
copy paste path wherever required
9.to check output there are 2 ways :

a.check output from browser --- open firefox browser type in url
localhost:50070- browse filesystem choose output directory and open part-00000 file
b.use cat command
hadoop dfs -ls /output --displays contents of output directory
hadoop dfs -cat /output/part-00000 - displays contents of this file
Hint: In main Class change
class name in JobConf() function
class name in setMapperClass() function
class name in setReducerClass() function
main method change in run() with constructor name

Input:



Output: